

CONCEPTUAL DESIGN DOCUMENT: FINANCE ANALYTICS DASHBOARD

1. Project Goal

The goal of this project is to develop a fully functional, full-stack finance analytics dashboard. This user-facing web application will serve as a simulation of a FinTech or accounting tool, allowing users to explore and understand their financial activity through a combination of interactive data tables and insightful visualizations. The core purpose of the project is to demonstrate a comprehensive understanding of the web development stack, from a responsive and intuitive frontend UI to a robust backend API and a well-structured relational database.

Core Objective: Build a responsive, data-driven dashboard with dynamic tables, insightful visualizations, and a robust backend architecture.

The dashboard is planned to be structured around two primary views, accessible via a simple navigation system:

1. Transactions: This view will present all user transactions in an interactive tabular format. Leveraging the DataTables.js library, the table will feature:

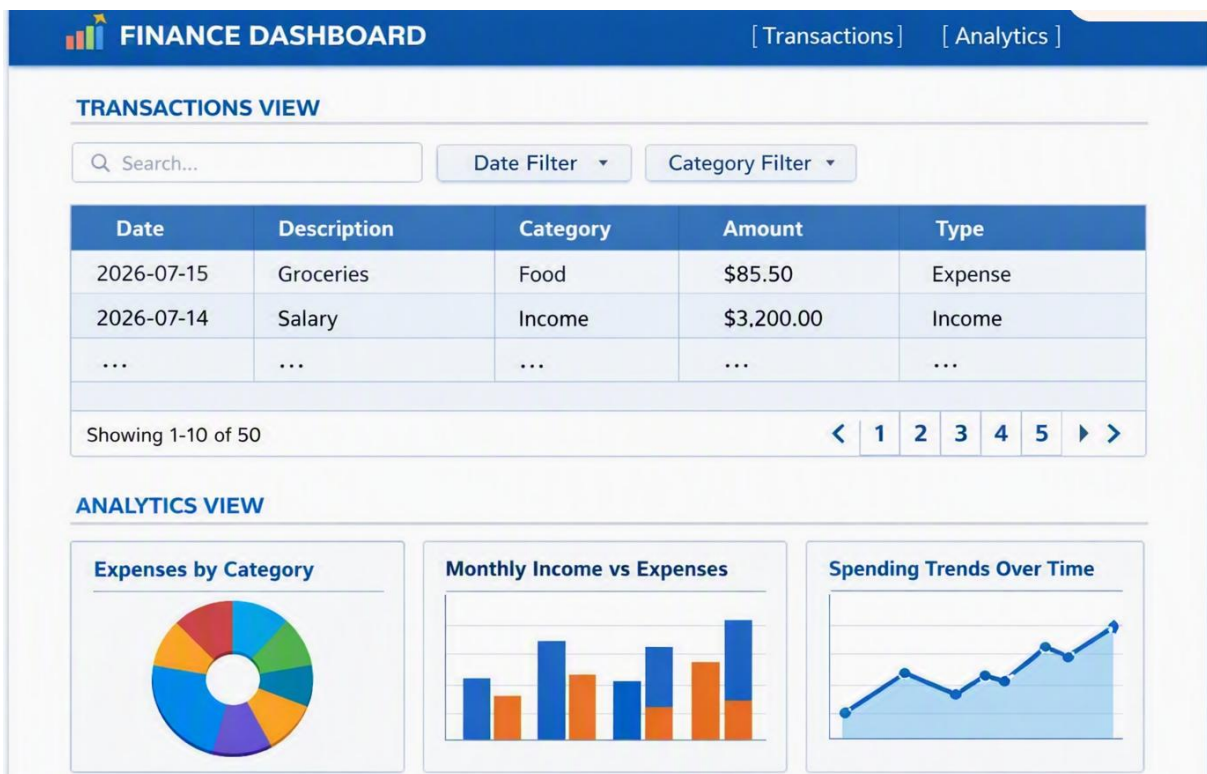
- o Pagination for managing large datasets, column sorting for order by date, category and amount, search and filtering for locating specific transactions, key fields such as date, description, category, amount and transaction ID

2. Analytics: This view will display a summarized analysis of financial data using interactive charts. Using Chart.js, the dashboard will implement at least the following three visualizations:

- o Expenses by Category (Pie Chart): To visually represent the proportion of spending across different categories (e.g., Food, Transport, Entertainment).
- o Monthly Income vs. Expenses (Bar Chart): To compare income and expenses over the last six to twelve months, showing net savings or deficits.
- o Spending Trends over Time (Line Chart): To track spending patterns over a selected period, helping identify trends and seasonality.

2. Dashboard Structure & UI Layout

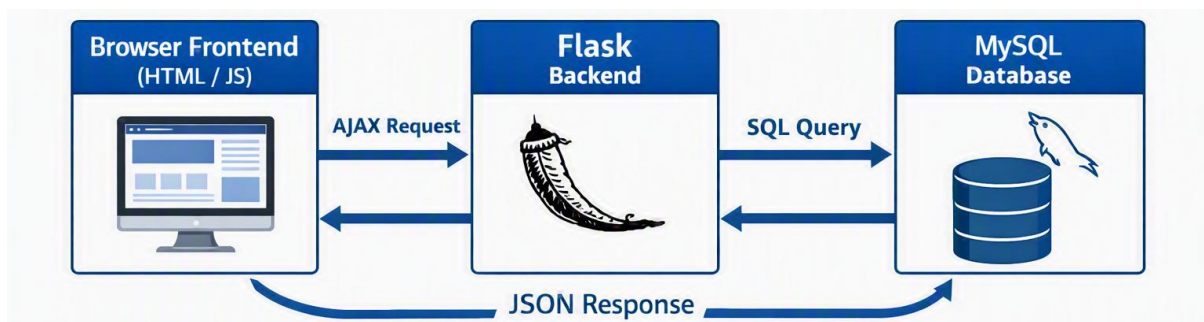
The dashboard features two primary views accessible via top navigation:



User Flow: Home (Transactions) → Filter/Search → View Details → Switch to Analytics → Explore Charts

3. Data Flow & Backend Architecture

The application follows a standard client-server architecture with RESTful APIs:



The application will use a client-server architecture. The frontend will request structured financial data through RESTful APIs provided by the Flask backend. The backend will query the MySQL database and return JSON data for rendering by DataTables.js and Chart.js.

The transaction endpoint will support server-side pagination, sorting, and filtering.

All endpoints will return JSON. Backend validation will ensure that income transactions are assigned only to income-type categories and expense transactions only to expense-type categories

API Endpoints with Parameters:

Endpoint	Method	Query Parameters	Description
/api/transactions	GET	?page=1&limit=25&sort=date&order=desc&category=food&type=expense	Fetch paginated, filtered, sorted transactions
/api/category-summary	GET	?month=2026-07	Aggregated spending by category
/api/monthly-summary	GET	?months=6	Monthly income vs expense totals
/api/trends	GET	?period=6&type=expense	Line chart data over time

4. Database design and entity relationship diagram (ERD)

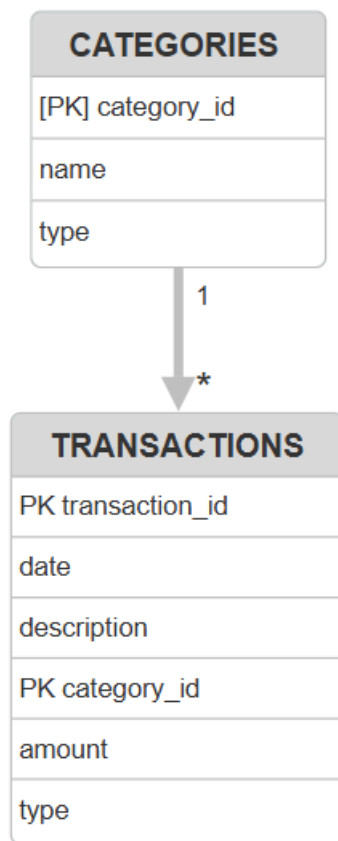
Relational MySQL database with two tables

Categories table:

Field	Type	Description
id	INT (PK, AI)	Unique identifier
name	VARCHAR(50)	Category name (e.g., Food, Transport)
type	ENUM('income','expense')	Ensures category matches transaction type

Transactions table:

Transactions TableField	Type	Description
id	INT (PK, AI)	Unique identifier
date	DATE	Transaction date
description	VARCHAR(255)	Transaction description
category_id	INT (FK)	References categories(id)
amount	DECIMAL(10,2)	Monetary value
type	ENUM('income','expense')	Transaction type



5. Summary calculations

Summary calculations; All summaries(monthly tools, category aggregates, trends) are calculated dynamically via SQL queries using SUM() , GROUP BY and DATE_FORMAT() functions. (). This will provide monthly income and expenses, spending by category, spending trends, and net savings based on the current transaction data. However, a small description on how summaries are calculated is included below:

Summary Type	SQL Method	Frequency
Monthly Income vs Expenses	SUM() + CASE + GROUP BY	Each API call
Expenses by Category	SUM() + GROUP BY + JOIN	Each API call
Spending Trends	SUM() + GROUP BY + DATE_FORMAT	Each API call
Net Savings	SUM(income) - SUM(expenses)	Calculated from summary

Example SQL for monthly summary:

```
SELECT
    DATE_FORMAT(date, '%Y-%m') as month,
    COALESCE(SUM(CASE WHEN type = 'income' THEN amount ELSE 0 END), 0) as total_income,
    COALESCE(SUM(CASE WHEN type = 'expense' THEN amount ELSE 0 END), 0) as total_expenses
FROM transactions
WHERE date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
GROUP BY DATE_FORMAT(date, '%Y-%m')
ORDER BY month DESC;
```

Example SQL for category summary:

```
SELECT
    c.name as category,
    SUM(t.amount) as total_spent
FROM transactions t
JOIN categories c ON t.category_id = c.id
WHERE t.type = 'expense'
    AND MONTH(t.date) = MONTH(CURRENT_DATE())
    AND YEAR(t.date) = YEAR(CURRENT_DATE())
GROUP BY c.id, c.name
ORDER BY total_spent DESC;
```

6. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript
CSS Framework	Bootstrap 5
Tables	DataTables.js
Charts	Chart.js
Backend	Python 3 + Flask
Database	MySQL
Version Control	Git & GitHub
Testing	Postman

7. Expected Features

1. Interactive Data Table (pagination, sorting, search, filtering)
2. Three dynamic charts (Pie: expenses by category, Bar: monthly income vs expenses, Line: spending trends)
3. Seeded sample data
4. RESTful API with pagination and filtering parameters
5. Responsive design (Bootstrap)
6. AJAX-driven dynamic updates (no page reloads)
7. Backend validation for category-transaction type matching
8. Dynamic SQL aggregation for all summaries

GitHub: <https://github.com/siza-mpofu/Finance-analytics-dashboard>